SRE Lead

Interview Preparation Handbook

119 questions across 14 topic areas

Site Reliability Engineer — LEAD posting
Platform ownership · 100% uptime architecture · blameless incident management
SLO/SLA/error-budget leadership · team mentoring & hiring · release automation

---

Positioning notes: Lead with the Barclays / Russell Investments banking background and the Terawe (Azure Synapse), Unique Vacations (Sandals), and RSAB engagements — real, verified metrics (40% alert-noise reduction, 35% MTTD improvement, 60% manual-deployment reduction, 3 SPOFs eliminated / 45% recovery-time improvement) anchor the architecture-for-uptime and automation-narrative answers. AZ-104 / SC-300 / SC-100 double as dual-cloud fluency and a security/identity edge even though this JD doesn't name a cloud. Items marked *(framework)* are structures to answer with, not fabricated stories — plug in your own real example where indicated.

Table of Contents

Role & Positioning	8 questions
Middleware (Weblogic/Tomcat/JBoss)	11 questions
Databases (Postgres/Mongo/MySQL)	10 questions
SLO/SLA & Error Budgets	8 questions
Team Management, Mentoring & Hiring	10 questions
CI/CD & Release Automation	10 questions
Monitoring & Logging	8 questions
Distributed Systems (RabbitMQ/Kafka/Redis)	10 questions
Docker & Kubernetes	8 questions
Incident Management & Postmortems	8 questions
High Traffic & Architecture for Uptime	8 questions
Linux & Shell Scripting	6 questions
JIRA & Quality Analysis	6 questions
Leadership & Behavioral	8 questions

Role & Positioning

POSITIONING & NARRATIVE

☐ **This is a Lead-level SRE role that also expects you to help select and mentor the team. How do you talk about that when your title so far has been Senior SRE/DevOps Engineer, not people-manager?**

Frame it as scope, not title. Reuse the ownership-depth reframe: "I've operated as the technical backbone for junior engineers even without a formal reporting line — reviewing runbooks, pairing on incident response, being the escalation point when things went past L2. What's new at Lead level is formalizing that: owning SLOs for the platform, sitting in on hiring, being accountable for the team's technical growth, not just my own output." *Bring your own:* a specific instance where you mentored or unblocked a teammate.

☐ **Walk me through your background in 2 minutes, tailored to this JD.**

Structure: (1) 14+ years across banking/finance (Barclays, Russell Investments) and enterprise platforms (Terawe/Microsoft, Unique Vacations/Sandals) — anchor with the verified metrics (40% alert-noise reduction, 35% MTTD improvement, 60% manual-deployment reduction, 3 SPOFs eliminated / 45% recovery-time improvement). (2) Deep hands-on AWS, Kubernetes, and Terraform. (3) Azure certifications (AZ-104/SC-300/SC-100) giving dual-cloud fluency and a security/identity edge. (4) Close on why this role: architecture for 100% uptime, team mentoring, and release-automation ownership are things you've already done informally and want to own formally.

☐ **Why are you moving from Senior SRE/DevOps Engineer into a Lead role now?**

Framework: natural progression — the technical depth is already there; what's missing is formal scope over SLOs/error budgets and people. *Bring your own:* a concrete example of already doing Lead-adjacent work (owning a postmortem process, being the de facto escalation point, mentoring).

☐ **Your certifications are Azure-heavy (AZ-104, SC-300, SC-100) but this JD doesn't mention Azure at all — how is that relevant?**

Position it as dual-cloud fluency plus a security/identity edge: SC-300 and SC-100 are identity and cloud-security-architecture certifications, which strengthens the JD's security line even if the underlying stack is AWS or on-prem. If the target environment's cloud isn't clear from the JD, it's fair to ask rather than assume.

☐ **This JD lists Weblogic, Tomcat, and JBoss — much older enterprise middleware than the cloud-native tooling on your resume. How do you address that gap honestly?**

Don't overclaim depth you don't have. Acknowledge that recent hands-on time has been in containerized/cloud middleware, but the underlying Java-app-server concepts (JVM tuning, thread pools, connection pooling, deployment/clustering/failover) transfer directly — and ask a clarifying question about how much of the target environment is still on that stack versus migrated.

☐ **What does 'production ownership' mean to you as a Lead, versus as a Senior Engineer?**

As a Senior, you execute against someone else's SLOs and runbooks and fix what breaks. As a Lead, you define the SLOs and error budgets, decide what 'good' looks like for the platform, own the postmortem process and hold peers accountable to it, and represent reliability in planning and architecture conversations before code ships — not just after an incident.

☐ **Give a 3-minute pitch of one project you'd point to as your best evidence for this role.**

Structure: context → what was fragile → what you built → measurable outcome. The RSAB chaos-engineering/SPOF story (3 SPOFs identified, 45% recovery-time improvement) is the strongest fit here since it's architecture-for-uptime work, matching this JD's 'co-create architecture for 100% uptime' line directly.

☐ **What's your approach to representing SRE in architecture and planning reviews, not just firefighting?**

Framework: get invited early (shift-left), bring a reliability checklist to design reviews (SPOFs, capacity headroom, rollback plan, alerting coverage), and push back with error-budget language rather than opinion — e.g. 'we're already burning 60% of this quarter's budget, this feature adds risk' — rather than blocking outright.

Middleware (Weblogic/Tomcat/JBoss)

APPLICATION SERVERS & MIDDLEWARE

☐ **What's the core difference between Tomcat, JBoss/WildFly, and WebLogic?**

Tomcat is a servlet/JSP container only — no full Java EE stack unless you add it (no built-in JMS, EJB, JTA). JBoss/WildFly and WebLogic are full Java EE/Jakarta EE application servers with built-in EJB, JMS, distributed transactions, and clustering. WebLogic is Oracle's commercial server, heavily enterprise-focused with deep Oracle DB integration, WLST scripting, and Node Manager; JBoss/WildFly is Red Hat's open-source equivalent, CLI-driven, running standalone or domain mode.

☐ **How do you troubleshoot a Tomcat instance that's slow to respond under load?**

Check thread pool exhaustion (maxThreads in server.xml, and a thread dump to see BLOCKED/WAITING threads), heap/GC pressure (jstat, GC logs), connector backlog (acceptCount), and downstream DB connection pool saturation. Correlate all of that with access-log response-time spikes and OS-level limits (netstat, file-descriptor ulimits).

☐ **What is a WebLogic domain, and what's the difference between the Admin Server and Managed Servers?**

A domain is a logically related group of WebLogic server instances managed as one unit. The Admin Server holds the config repository and admin console; Managed Servers run the actual applications and are configured via the Admin Server but keep a cached copy of config, so they can keep running applications even if the Admin Server itself goes down.

☐ **How would you set up clustering/failover in JBoss or WebLogic for high availability?**

JBoss: domain mode with multiple server groups behind a load balancer, using mod_cluster for session-affinity-aware failover. WebLogic: a cluster of Managed Servers with in-memory session replication (or a shared session store) front-ended by a load balancer, with Node Manager providing automatic server restart.

☐ **What's a thread dump and how do you use it to diagnose a hung Java app server?**

Capture via jstack <pid> (or kill -3, which writes to the server log). Take 2–3 dumps a few seconds apart and look for threads stuck in the same stack trace across all of them — that's a real hang (deadlock, DB wait, lock contention), whereas differing stacks each time just indicate normal transient activity.

☐ **How do you handle a JBoss/WebLogic deployment that fails partway through, leaving the app in an inconsistent state?**

Roll back to the last known-good archive, check the deployment scanner logs and deployment-plan status for where it stalled, verify there are no orphaned MBeans or stuck deployment units, and — for WebLogic specifically — check the 'prepared' vs 'activated' two-phase deployment state before retrying.

☐ **What's connection pooling and why does misconfiguring it cause outages?**

A managed pool of reusable DB connections between the app server and the database, avoiding the cost of a fresh TCP-plus-auth handshake per request. Misconfiguration is one of the most common root causes of app-server 'hangs' that are actually connection-pool waits: too small a pool starves threads waiting for a connection under load; too large a pool can exhaust the database's own max-connections limit.

☐ **How do you approach migrating an app off WebLogic/JBoss to a container/K8s-native stack without downtime?**

Strangler-fig pattern: stand up the containerized version alongside the legacy app server, route a small percentage of traffic to it via the load balancer or API gateway, validate parity on logs/error rates/latency, progressively shift traffic, and keep the legacy path as a rollback target until confidence is high before decommissioning it.

☐ **What monitoring do you put in front of a legacy WebLogic/JBoss estate that has no built-in Prometheus exporter?**

JMX is the integration point — use a Prometheus JMX exporter or an APM agent (AppDynamics, which this JD already calls out) to pull heap, thread-pool, GC, and deployment metrics into the same dashboards as the rest of the estate, rather than maintaining a separate monitoring stack for the legacy tier.

☐ **What's your approach to zero-downtime schema migrations on a live database?**

Expand/contract pattern: add new columns or tables without removing the old ones, deploy app code that writes to both, backfill, cut reads over to the new schema, then remove the old schema in a later release. Avoid long table locks by using online DDL tooling (pt-online-schema-change/gh-ost for MySQL) where a plain ALTER would block.

☐ **A JBoss standalone instance keeps going OOM in production. Walk through your diagnosis.**

Enable heap-dump-on-OOM (-XX:+HeapDumpOnOutOfMemoryError), analyze the dump with Eclipse MAT for the dominator tree, and check for common Java-app-server leak patterns (ThreadLocal not cleared, static collections growing unbounded, unclosed resources). Correlate with deployment history — did a recent release introduce this? — before assuming it's purely a heap-sizing problem versus a genuine code leak.

Databases (Postgres/Mongo/MySQL)

DATABASE OPERATIONS FOR SRE

☐ **As an SRE, what's your role with databases even if you're not a DBA?**

Not schema or query design — but you own availability: replication health, failover mechanics, backup/restore validation, connection-pool tuning from the application side, capacity and disk-growth trending, and being the first responder when a DB-related alert fires, before escalating to a DBA.

☐ **Explain Postgres streaming replication and how failover works.**

The primary streams WAL (write-ahead log) to one or more standbys via a replication slot. Failover — manual, or automated via a tool like Patroni or repmgr — promotes a standby to primary and requires updating application connection strings or DNS. Any transactions not yet shipped at failover time are lost, which is the real RPO cost of asynchronous replication; synchronous replication trades write latency for zero data loss.

☐ **How do you monitor MySQL/Postgres replication lag and what do you do when it spikes?**

Postgres: pg_stat_replication (replay_lag). MySQL: SHOW SLAVE STATUS (Seconds_Behind_Master). Spikes usually mean the replica can't keep up with write volume (I/O bound) or a long-running query is blocking replay — check replica disk I/O first, then kill any blocking query, and scale replica hardware if this is a recurring pattern rather than a one-off.

☐ **What's the difference between MongoDB replica sets and sharding, and when do you need each?**

A replica set copies the same data across nodes for HA and read-scaling — one primary, secondaries, automatic failover via election. Sharding horizontally partitions data across shards for write-scaling or dataset size beyond what one node can hold. You need sharding once dataset size or write throughput outgrows a single replica set, not just for high availability.

☐ **How would you diagnose a MySQL/Postgres instance that suddenly has high CPU?**

Check active queries (SHOW PROCESSLIST / pg_stat_activity) for long-running or newly-slow queries — often caused by a missing index after a schema change, or a bad query plan from stale statistics. Also check for lock contention, and correlate the spike with recent deploys or a genuine traffic increase.

☐ **What's your backup/restore validation practice — how do you know your backups actually work?**

A backup isn't proven until it's restored. Schedule periodic restore-to-a-scratch-instance drills rather than just checking that the backup job reported success, track restore time against your RTO target, and alert on backup failures with the same severity as a production incident.

☐ **Connection pool exhaustion is a common outage cause — how do you prevent and detect it?**

Size the app-side pool (HikariCP, PgBouncer, etc.) so pool-size × number-of-app-instances stays under the database's max_connections. Monitor pool wait time and active/idle connection counts, and use PgBouncer/ProxySQL as a connection multiplexer in front of Postgres/MySQL as you scale out app instances.

☐ **How do you handle a 'noisy neighbor' query that's degrading performance for everyone on a shared DB instance?**

Identify it via the slow-query log or pg_stat_statements, work with the owning team to add an index or rewrite the query, and use resource governance as a stopgap — query timeouts, or moving reporting workloads to a read replica — to isolate it from the primary transactional path.

☐ **What's your approach to zero-downtime schema migrations on a live database?**

Expand/contract: add new schema elements without removing old ones, deploy app code that writes to both, backfill, cut reads over, and remove the old schema in a later release — avoiding long table locks with online DDL tooling where a plain ALTER would block for too long.

☐ **How do you decide RPO/RTO targets for a database, and how does that shape your replication/backup design?**

RPO and RTO come from the business, not the infra team — ask how much data loss is acceptable and how fast the business needs to be back up. Near-zero RPO pushes toward synchronous, multi-AZ replication; an RTO measured in minutes pushes toward automated failover (Patroni, RDS Multi-AZ) rather than a manual, runbook-driven recovery.

SLO/SLA & Error Budgets

STRATEGIC RELIABILITY LEADERSHIP

☐ **How do you define an SLO for a service that's never had one?**

Start from user-facing behavior, not infra metrics: pick 1–2 SLIs that reflect what users actually experience (availability = % successful requests, latency = % requests under a threshold), set the SLO based on historical performance plus some headroom, and revisit after a quarter of real data rather than guessing perfection upfront.

☐ **What's the difference between an SLA and an SLO, and why does it matter who owns each?**

The SLO is internal — the target the team engineers to. The SLA is external and contractual — what's been promised customers, usually looser than the SLO, with financial or legal consequences if breached. The SLO should always be stricter than the SLA, so the team has room to react before an actual contract breach.

☐ **How do you use an error budget to make a real go/no-go decision, not just as a dashboard number?**

If the budget's healthy, ship at normal velocity, including riskier changes. If it's nearly exhausted, freeze non-critical releases, prioritize reliability work, and require extra review on anything touching the affected service. The error budget converts 'reliability' from a vague value into a number engineering and product both respect.

☐ **A product team wants to ship a feature that risks breaching this quarter's error budget. How do you have that conversation?**

Don't say no unilaterally — bring the burn-rate data, show the trend, and offer options: ship behind a feature flag/canary with a fast rollback plan, delay to the next budget cycle, or accept the risk with explicit sign-off from product leadership. The point of the error budget is making the tradeoff visible and shared, not giving SRE a veto.

☐ **How do you set SLOs across multiple regions/environments for a globally-scaled service?**

Per-region SLOs if regions can fail independently and traffic is regionally routed; a single global SLO if the service presents as one logical experience to users regardless of region. The choice follows your failure and routing architecture, not a fixed rule.

☐ **What's burn-rate alerting and why is it better than a static threshold alert on the SLO itself?**

A burn-rate alert fires on how fast the error budget is being consumed (e.g., 'at this rate we exhaust 30 days of budget in 2 hours') rather than waiting for the SLO itself to be breached — giving early warning while budget still remains, using multiple time windows so a short window catches fast-burn incidents and a longer window avoids noise from brief blips.

☐ **How do you get a legacy system with no historical SLI data to a mature SLO practice?**

Instrument first, even with basic request-success and latency logging. Observe for a baseline period before setting any target, start with a loose SLO you're confident you can hit, and tighten gradually — setting an aggressive SLO on day one with no data just produces immediate, meaningless breaches.

☐ **As a Lead, how do you keep SLOs from becoming a purely technical exercise the business ignores?**

Tie SLOs to metrics the business already tracks (customer complaints, churn, revenue-impacting incidents), report error-budget status in the same cadence and forum as other business metrics, and translate breaches into business language — 'we had 40 minutes of budget-affecting downtime, here's the estimated customer impact' — rather than raw percentages.

Team Management, Mentoring & Hiring

BUILDING & LEADING THE TEAM

☐ **This JD asks you to help select the team. What do you look for when interviewing an SRE candidate?**

Framework: troubleshooting methodology over memorized answers — walk them through diagnosing an ambiguous incident rather than asking 'what's a CrashLoopBackOff'; evidence of ownership beyond their ticket queue; and communication under pressure, judged by how clearly they explain a past incident, including their own mistakes in it. *Bring your own:* a specific technical scenario you'd actually ask.

☐ **How do you structure mentoring for a junior engineer joining your on-call rotation?**

Shadow rotation first — they observe, you're primary. Then reverse shadow — they're primary, you're backup and review their actions post-incident. Pair that with a clear escalation path and a runbook they help maintain, rather than just handing over the pager.

☐ **How do you evaluate performance for someone on your team — 'on what basis'?**

Framework: not just uptime or ticket count — those are team outcomes, not individual signal. Look at ownership (do they proactively surface risks, or only react), the quality of their incident write-ups, growth in scope over time, and peer/cross-team feedback, weighed together rather than any single metric.

☐ **Tell me about handling an underperforming engineer on your team.**

Framework: a private, specific conversation first — behavior and impact, not character judgment — followed by a written improvement plan with concrete, measurable expectations and a timeline. Regular check-ins rather than a single pass/fail moment at the end, and documentation throughout so the outcome is fair and defensible either way. *Bring your own* if you have a real example.

☐ **How do you build a 'high-performing SRE group' as this JD phrases it, from a mixed-experience team?**

Framework: pair junior with senior on real incidents, not just training exercises; rotate on-call fairly so expertise doesn't bottleneck on one person; invest in shared tooling and runbooks so knowledge isn't locked in people's heads; and protect toil-reduction time explicitly rather than always letting it lose to feature deadlines.

☐ **How do you handle disagreement with a peer or your own manager about a reliability tradeoff?**

Bring data, not opinion — error-budget burn, incident history, capacity headroom. State the tradeoff explicitly, and once you've been heard and the decision is made, commit to it rather than relitigating it in every subsequent meeting — but document the risk you flagged so it isn't lost.

☐ **What's your approach to knowledge-sharing so the team isn't dependent on one 'hero' engineer?**

Runbooks written during the incident, not after. A rotating incident-commander role so it isn't always the same person. Blameless postmortems used as teaching material for the whole team, not just the responder's private write-up.

☐ **How do you keep a team motivated through a stretch of heavy on-call load or repeated incidents?**

Framework: acknowledge the load honestly rather than minimizing it, push visibly for toil-reduction work to actually get prioritized rather than just telling the team to endure it, rotate fairly, and use postmortems to fix root causes so the same page doesn't keep recurring — motivation follows from people seeing the load actually decrease, not from reassurance alone.

☐ **How would you onboard a new hire onto a platform with 5+ years of undocumented tribal knowledge?**

Pair them with a buddy for their first incidents rather than a document dump. Have them write or update runbooks as they learn — fresh eyes catch gaps a five-year veteran no longer notices. Treat every 'wait, why does it work that way' question as a documentation gap to close, not just answer verbally.

☐ **Describe your leadership style in one sentence, then back it up.**

Framework: pick something concrete and defensible rather than a generic adjective like 'collaborative' — for example, 'lead by being the calmest person in the incident channel, and let the postmortem, not blame in the moment, drive the real fixes' — and back it with one real example.

CI/CD & Release Automation

JENKINS, ANSIBLE & RELEASE ENGINEERING

☐ **This JD calls out 'fundamental aspects for release automation: packaging, dependencies, promotion, deployment, compliance.' Walk through each.**

Packaging: build a versioned, immutable artifact (container image, jar, deb/rpm) once. Dependencies: pin and scan them (lockfiles, SBOM) so the same build is reproducible and auditable. Promotion: the same artifact moves dev → staging → prod without rebuilding, gated by tests and approvals at each stage. Deployment: the actual rollout mechanism — blue-green, canary, or rolling. Compliance: change records, approvals, and an audit trail satisfying whatever regulatory regime applies — usually heaviest in banking.

☐ **How do you design a Jenkins pipeline for a regulated environment that needs a full audit trail?**

Declarative pipeline-as-code checked into version control, so the pipeline definition itself is auditable. Mandatory approval gates before prod stages. Artifact provenance — knowing exactly which commit and build produced the deployed artifact. Pipeline logs retained and tied to a change ticket (JIRA/ServiceNow), not left only in Jenkins's own history.

☐ **What's your approach to using Ansible for configuration management vs. immutable infrastructure?**

Ansible is strong for correcting configuration drift and orchestrating deployments across existing hosts — especially for the WebLogic/JBoss/Tomcat middleware this JD mentions, which isn't always containerized. For cloud-native workloads, prefer immutable images (built via Packer or container builds) over configuring live hosts, and use Ansible for orchestration — sequencing a rolling restart, running health checks — rather than fighting drift on long-lived servers.

☐ **A green pipeline just deployed a broken release to production. Why does this happen and how do you prevent it?**

Green means the pipeline's own checks passed, not that the app is healthy — usually a gap between what's tested in a lower environment and real production conditions (data volume, downstream dependencies, config differences). Prevent it with post-deploy smoke tests or synthetic checks as a pipeline stage, canary analysis before full rollout, and rollback triggers tied to real production signals rather than just pipeline exit codes.

☐ **How do you handle secrets in a CI/CD pipeline without leaking them into logs or version control?**

Never in the repo or in plaintext pipeline config — use a secrets manager (Vault, AWS Secrets Manager, Jenkins Credentials Binding) injected at runtime, mask known secret patterns in log output, and rotate anything that's ever been accidentally exposed rather than assuming a deleted log entry is sufficient.

☐ **What's your promotion strategy for artifacts across environments — rebuild per environment, or build once?**

Build once, promote the same artifact. Rebuilding per environment risks subtle differences — dependency resolution at a different time, a different base-image patch level — between what was tested and what was shipped. Environment-specific config should be externalized (env vars, config maps), not baked in at build time.

☐ **How do you roll back a bad release safely and quickly?**

Depends on strategy: blue-green — flip traffic back to the still-running old version, near-instant; canary — halt promotion and drain the canary; rolling — redeploy the previous known-good artifact version. The precondition across all of them is keeping the previous artifact readily deployable and testing rollback before you need it live, not improvising it during a real incident.

☐ **What's your view on shell scripting in an automation pipeline — when is it appropriate vs. a real IaC/config tool?**

Fine for glue logic, one-off diagnostics, and simple orchestration steps. Once a script grows conditionals, retries, or state that needs to survive across runs, it belongs in a proper tool — Ansible, Terraform, or a real pipeline stage. Shell scripts that grow organically into 'the deployment system' are a common source of untestable, undocumented fragility.

☐ **How do you handle dependency vulnerabilities discovered in an artifact already in production?**

Triage by actual exploitability and exposure — is the vulnerable code path reachable, is the service internet-facing — rather than patching every CVE with equal urgency. Patch and re-promote through the same pipeline, not a manual hotfix that bypasses the process, and track remediation against SLAs by severity so this doesn't silently linger.

☐ **What compliance evidence would you show an auditor for 'who approved this production deployment, and why'?**

A linked chain: JIRA/change ticket → pipeline run ID → approver identity and timestamp from the pipeline's approval gate → deployed artifact hash/version — every link traceable without relying on someone's memory or a Slack message.

Monitoring & Logging

NAGIOS / APPDYNAMICS / ELK / PROMETHEUS

☐ **This JD lists Nagios alongside AppDynamics and ELK/Prometheus. How do these differ and where does each fit?**

Nagios: classic host/service up-down and threshold checking — simple and reliable, but limited on rich time-series and dashboarding. Prometheus: pull-based time-series metrics with powerful querying (PromQL) and native alerting via Alertmanager, a better fit for cloud-native/K8s. AppDynamics: commercial APM giving deep application-level tracing — which method or DB call is slow — beyond infra-level metrics. ELK: log aggregation and search, not metrics, complementing both by letting you correlate an alert with the actual error text.

☐ **How would you migrate an estate monitored with Nagios toward a modern stack without losing coverage during the transition?**

Run both in parallel rather than a hard cutover — port checks one service at a time, validate the new stack catches the same incidents the old one would have (replay recent real incidents against new alert rules), and only decommission Nagios checks once equivalent or better coverage is proven for that service.

☐ **What makes a good alert vs. a noisy one, and how do you reduce alert fatigue?**

Good alerts are actionable — someone needs to do something right now — and tied to user impact (SLO-based), rather than every infra threshold crossing. Reduce noise by consolidating related alerts into one page per incident, tuning thresholds against real historical baselines, and routing low-urgency signals to a dashboard or ticket instead of a page.

☐ **How do you design an ELK-based logging pipeline for a high-traffic system without it falling over or costing a fortune?**

Structured logging at the source (JSON, not free text) so parsing is cheap. Index lifecycle management — hot/warm/cold tiers, aggressive retention on verbose debug logs. Sample or filter high-volume, low-value log lines (health-check noise) before they hit Elasticsearch at all, rather than ingesting everything and filtering at query time.

☐ **How do you use AppDynamics/APM data during an active incident, not just for post-incident analysis?**

Use the transaction-snapshot or call-graph view to immediately see which downstream dependency is slow — a database, an external API, a specific method — rather than guessing from infra metrics alone. This usually cuts triage time significantly versus starting from host-level CPU/memory graphs.

☐ **What's the difference between the RED method and the USE method for building dashboards, and when do you use each?**

RED (Rate, Errors, Duration) is service/request-centric — best for user-facing services. USE (Utilization, Saturation, Errors) is resource-centric — best for infrastructure. Use RED dashboards to know if the service is healthy for users, and USE dashboards once you're triaging to find out why.

☐ **How do you set up synthetic monitoring/health checks, and how is that different from watching real traffic metrics?**

Synthetic checks proactively simulate user transactions on a schedule, catching issues even during low-traffic periods — overnight, for example — when real-traffic metrics wouldn't yet show a problem. Real-traffic metrics tell you about impact already happening; synthetics can catch a break before real users hit it.

☐ **A dashboard shows everything green but users are reporting problems. How do you approach that gap?**

A classic monitoring blind spot. Check what's not instrumented — a specific customer segment, a third-party dependency, a code path monitoring doesn't cover. Verify alert thresholds actually match user-impacting behavior — green might just mean 'within threshold' while still degraded. Treat the user report as a signal to add missing instrumentation, not something to dismiss because the dashboards disagree.

Distributed Systems (RabbitMQ/Kafka/Redis)

MESSAGING, STREAMING & IN-MEMORY STORES

☐ **When would you choose RabbitMQ over Kafka, or vice versa?**

RabbitMQ: a traditional message broker, strong for complex routing (exchanges/queues/bindings), per-message acknowledgment, and moderate-throughput task-queue workloads. Kafka: a distributed log built for very high-throughput, durable, replayable event streams with multiple independent consumers reading the same data at their own pace. Choose RabbitMQ for request/task-queue patterns, Kafka for event-streaming architectures needing replay or many consumers.

☐ **How do you troubleshoot a growing RabbitMQ queue backlog?**

Check consumer count and health — are consumers down or slow — check per-message processing time for a slow downstream dependency, verify the prefetch count isn't starving consumers, and scale consumers horizontally as immediate mitigation before the queue hits memory/disk limits and the broker itself becomes unstable.

☐ **What's Kafka consumer lag and how do you diagnose/fix it?**

Lag is the difference between the latest produced offset and the consumer group's committed offset — the consumer is falling behind. Diagnose via consumer-group lag metrics (kafka-consumer-groups.sh or a Prometheus exporter). Fix by scaling consumer instances up to the partition count, optimizing per-message processing time, or adding partitions if the topic is under-partitioned for the throughput.

☐ **How does Kafka guarantee message durability, and what's the tradeoff with acks configuration?**

Messages are replicated across brokers per the topic's replication factor. acks=0 is fire-and-forget — fastest, least durable. acks=1 waits for the leader only — can lose data if the leader fails before replicating. acks=all/-1 waits for all in-sync replicas — strongest durability, higher latency. Choose based on whether the data is replayable/non-critical or must never be lost.

☐ **What's Redis used for in a typical architecture, and what are the risks of treating it as a primary datastore?**

Typically caching, session storage, rate-limiting counters, and pub/sub — fast and in-memory. Treating it as primary storage is risky: default persistence (RDB snapshots/AOF) trades some durability for speed, and a restart or failover can lose recent writes depending on the persistence config — fine for a cache backed by a source of truth elsewhere, risky if Redis is the only copy of important data.

☐ **How do you handle Redis failover in production — what's the SRE's role in that?**

Redis Sentinel (single-primary setups) or Redis Cluster (sharded setups) handle automatic failover. The SRE's role is ensuring clients reconnect to the new primary automatically — via Sentinel-aware or cluster-aware client libraries — monitoring failover events, and understanding that failover can cause a brief write-unavailability window and potential data loss for the most recent writes.

☐ **What causes a Kafka broker to become the bottleneck, and how do you scale around it?**

Common causes: too few partitions for the topic, limiting parallelism; uneven partition leadership overloading one broker; or under-provisioned disk I/O/network. Scale by adding partitions carefully (they can't easily be reduced later), rebalancing leadership across brokers, and adding broker capacity ahead of sustained growth rather than reactively.

☐ **Explain the CAP-theorem tradeoff as it applies to Redis Cluster or a Kafka partition during a network split.**

During a network partition, you choose consistency — reject writes/reads that can't be confirmed against the full quorum — or availability — keep serving from whatever's reachable, risking stale or conflicting data. Kafka favors consistency for writes to a given partition (only the leader accepts writes; followers must catch up). Redis Cluster can briefly serve stale reads from a partitioned node depending on configuration.

☐ **How would you design a rate-limiter using Redis for a high-traffic API?**

A sliding-window or token-bucket counter per client key with a TTL, incremented atomically — Redis INCR plus EXPIRE, or a Lua script for atomicity across both operations — checked before the request is processed. Redis's single-threaded atomic operations make this a natural fit without needing extra locking.

☐ **A message queue (RabbitMQ or Kafka) is a single point of failure in the architecture diagram you're reviewing. How do you push back constructively?**

Use this JD's own 'architecture for 100% uptime' language directly — quantify the blast radius of that one component being down, propose the standard mitigation (RabbitMQ mirrored/quorum queues across nodes, or a Kafka cluster with a replication factor of 3+ across AZs), and treat it as a design-review finding to fix before launch rather than an incident to survive later.

Docker & Kubernetes

CONTAINERS & ORCHESTRATION FUNDAMENTALS

☐ **What's the difference between a Docker image and a container, in your own words for someone less technical?**

An image is the read-only blueprint — like a class. A container is a running instance of that blueprint — like an object. You can run many containers from the same image, each with its own writable layer and process space.

☐ **How do you keep container images small and secure?**

Multi-stage builds — build tools in one stage, only the final artifact copied to a minimal runtime base like distroless or alpine. Pin base image versions instead of :latest. Scan images in the pipeline (Trivy/Grype) before they're promoted to the next environment.

☐ **What's a Kubernetes Deployment vs. a StatefulSet, and why does it matter for a database vs. a stateless API?**

A Deployment manages interchangeable, stateless pod replicas — any pod can be replaced by any other. A StatefulSet gives each pod a stable identity and persistent storage tied to that identity — pod-0 always gets the same PVC. That's required for anything where identity, order, or storage-per-instance matters, like a database cluster.

☐ **A pod is stuck in CrashLoopBackOff — walk through your triage.**

kubectl describe pod for events — OOMKilled, failed probes, image-pull errors. kubectl logs --previous for the crashed container's last output before restart. Check resource requests/limits for OOMKilled specifically, and check whether a dependency needed at startup — a database, config — is actually reachable.

☐ **What's the purpose of readiness vs. liveness probes, and what happens if you configure them wrong?**

Liveness answers 'is this container alive — restart it if not.' Readiness answers 'is this container ready to serve traffic — remove it from the Service endpoints if not.' An overly aggressive liveness probe causes unnecessary restart loops on a slow-but-healthy app; a missing or too-permissive readiness probe sends traffic to a pod that isn't actually ready yet, causing user-facing errors during startup or deploys.

☐ **How do resource requests and limits affect scheduling and stability?**

Requests are what the scheduler reserves capacity for on a node. Limits cap actual usage — hitting a CPU limit throttles the container, hitting a memory limit gets it OOMKilled. Under-setting requests risks overcommitting a node; omitting limits risks one noisy pod starving others sharing that node.

☐ **How would you approach a namespace-wide outage where every pod in one namespace suddenly can't reach the database?**

Check for a shared cause first — a NetworkPolicy change, a Service/Endpoint misconfiguration, CoreDNS resolution failure, or the database itself being down or at its connection limit — before assuming every individual pod broke independently. A namespace-wide symptom usually has a namespace- or cluster-level cause.

☐ **What's your approach to capacity planning for a Kubernetes cluster ahead of an expected traffic spike?**

Load-test against realistic traffic patterns beforehand, verify the horizontal pod autoscaler's thresholds and max replica count are actually sufficient — not just that HPA is enabled — confirm Cluster Autoscaler can add node capacity fast enough for the spike's ramp time, and pre-warm anything with a slow cold start rather than relying purely on reactive autoscaling.

Incident Management & Postmortems

BLAMELESS INCIDENT RESPONSE

☐ **Walk through your incident response process from alert to resolution.**

Alert fires → acknowledge and assess severity/impact → declare an incident and assign an incident commander if severity warrants → mitigate first — rollback or failover — before root-causing → communicate status to stakeholders on a set cadence → confirm resolution against the actual user-facing signal, not just the alert clearing → schedule the postmortem.

☐ **What makes a postmortem 'blameless' in practice, not just in name?**

The write-up focuses on system and process gaps — 'the alert threshold was too high,' 'the runbook didn't cover this case' — rather than individual actions. The test: would a different, equally competent person have made the same decision given the same information at the time? If yes, it's a system gap, not a person gap.

☐ **How do you handle a stakeholder who's frustrated and pushing for instant answers mid-incident?**

Separate the incident-response channel from stakeholder communication — a dedicated status update rather than pulling responders into a call mid-fix. Give a real cadence ('next update in 15 minutes') even if the update is 'still investigating,' and resist speculating on root cause before you actually know it — a wrong early guess is worse than 'we don't know yet.'

☐ **What's the difference between Incident, Problem, and Change management under ITIL?**

Incident: restore service now, minimize impact. Problem: find and fix the underlying root cause so it doesn't recur — can run after the incident is resolved. Change: the controlled process for making that fix, or any planned change, without introducing new risk. They're distinct processes because the urgency and rigor needed differ — incident is about speed, change is about control.

☐ **How do you prioritize incidents — what determines P1 vs. P2 vs. P3?**

Typically an Impact × Urgency matrix — impact is how many users or how much revenue is affected, urgency is how fast it's getting worse or how time-sensitive the business context is. A P1 usually means significant user-facing impact with no workaround, requiring immediate response regardless of time of day.

☐ **Describe a time an incident revealed a gap in your monitoring, not just a code bug. (framework)**

Structure: what broke → why existing alerts didn't catch it early → what you added afterward — a new alert, a dashboard panel, a synthetic check → how you verified the new coverage would have caught it. *Bring your own* real example, ideally anchored to something from Terawe, RSAB, or Unique Vacations.

☐ **How do you prevent the same root cause from recurring after a postmortem, beyond just writing action items?**

Action items need an owner and a due date, tracked like any other work item — not a document nobody revisits. Genuinely high-severity fixes should compete for sprint priority alongside features; a postmortem with action items that never get done is exactly why the same incident recurs.

☐ **How do you run a postmortem for an incident where your own team's decision made things worse in the moment?**

The same blameless standard applies to yourself — state plainly what you did and why it seemed right at the time, what information was missing that would have changed the decision, and what's changing — a runbook update, a new check — so the next person has that information rather than reproducing the same reasonable-at-the-time mistake.

High Traffic & Architecture for Uptime

SCALABILITY & 100% UPTIME DESIGN

☐ **What does 'co-creating architecture for 100% uptime' actually mean in practice, since true 100% is never achievable?**

In practice, it means eliminating single points of failure at every layer — compute, network, database, DNS — and setting an SLO as close to 100% as the business needs and can afford. '100%' in a JD is aspirational language for 'no single component failure should cause a customer-visible outage,' not a literal guarantee.

☐ **How do you design 'alternate connectivity' as this JD phrases it, for a critical system?**

Multiple independent network paths to the same destination — multi-AZ/multi-region networking, redundant ISP or Direct Connect links, DNS failover across regions — so no single network failure, like a fiber cut or an AZ network partition, takes down connectivity entirely. The key design principle is that redundant paths must not share a common failure point.

☐ **How do you approach capacity planning for a system you expect to 10x in traffic over the next year?**

Load-test against projected, not just current, traffic. Identify the component that will bottleneck first — usually a database or another stateful dependency before stateless compute — and build the scaling plan around that specific bottleneck rather than uniformly scaling everything.

☐ **What's the difference between horizontal and vertical scaling, and when do you actually still need vertical scaling?**

Horizontal — more instances — is generally preferred for resilience (no single bigger box to lose) and elasticity. Vertical still matters for stateful components that can't trivially shard, like a single-writer database primary or some legacy middleware — scale those up until sharding or read-replica strategies become viable, since horizontal scaling isn't free for stateful systems.

☐ **How do you handle a system with a hard dependency on a third-party API that occasionally has outages?**

A circuit breaker to fail fast rather than let requests pile up waiting on a dead dependency, a fallback or degraded-mode experience where possible — cached or stale data rather than a hard error — and timeout tuning so one slow dependency doesn't exhaust your own thread or connection pool.

☐ **What's your approach to launch review / go-live readiness for a new service before it takes production traffic?**

A checklist: SLOs defined; alerting and dashboards in place before launch, not added after the first incident; capacity tested against expected load plus headroom; rollback plan tested; on-call runbook written; and a clear owner. Apply the same rigor whether it's a brand-new service or a major architecture change to an existing one.

☐ **How do you think about blast radius when designing a change to a high-traffic system?**

Deploy to the smallest reasonable slice first — a single instance, a canary, one region — and expand only after validating real signals. Keep the ability to isolate or roll back that slice independently, and explicitly ask 'if this specific change is wrong, how many users does it affect and for how long' before shipping to everyone.

☐ **Describe a real high-traffic/high-scale system you've worked on and the reliability challenge that came with it. (framework)**

Structure: scale characteristics — traffic pattern, peak vs. average → the specific failure mode that scale created — something fine at low load, buckling at peak → what you changed → the measurable outcome. *Bring your own* real example, anchored to a verified metric where it applies — e.g. the 60% manual-deployment reduction, or the 45% recovery-time improvement.

Linux & Shell Scripting

OS TROUBLESHOOTING & AUTOMATION

☐ **A production server is at 100% CPU — walk through your first 5 minutes.**

top/htop to find the offending process, and check whether it's one runaway process or many, for load-average context. pidstat or strace if the process itself needs profiling. Check whether this correlates with a recent deploy or a traffic pattern, and decide quickly whether to mitigate — restart or scale — before fully diagnosing root cause if user impact is ongoing.

☐ **Disk is filling up on a production host — how do you find what's eating space and fix it safely?**

du -sh /* narrowing down directory by directory, or ncdu for interactive browsing. Check specifically for runaway log files or core dumps. Before deleting anything in production, verify it's genuinely disposable — rotated logs past retention, not a file still held open by a live process, since deleting an in-use file doesn't actually free space until that process closes it.

☐ **Write or describe a one-liner to find and kill all processes matching a pattern, safely.**

pgrep -f "pattern" to list and confirm the matches first — running pkill -f "pattern" directly without checking first risks matching something unintended. Prefer a targeted, PID-based kill once the list is confirmed.

☐ **How do you safely use awk/sed/grep to parse a large log file for an incident, without loading the whole thing into memory?**

Stream-process rather than load-and-search — grep/awk process line by line and don't require the whole file in memory. Narrow with grep first to cut volume before piping into more expensive awk/sed processing, and use zgrep directly on rotated/compressed logs rather than decompressing first.

☐ **What's the difference between a hard link and a symbolic link, and why does it matter for a deployment script?**

A hard link is another directory entry pointing to the same inode — same filesystem only, and it survives the original being deleted. A symlink is a separate file containing a path reference — it can cross filesystems but breaks if the target moves. Deployment scripts commonly symlink a 'current' release folder to the latest versioned release directory specifically so a rollback is just re-pointing the symlink, not recopying files.

☐ **How do you approach permissions/umask issues that cause a deployment to fail intermittently?**

Check the actual effective permissions on the failing path, not just what you expect from the deploy script. Verify the umask of the user or service account running the deployment — a default umask can silently produce files without expected group-write permissions. Make permission requirements explicit in the deployment script (chmod/chown) rather than relying on inherited defaults that can vary by host.

JIRA & Quality Analysis

PROJECT & QUALITY TRACKING

☐ **How do you use JIRA specifically to track reliability work, not just feature delivery?**

A dedicated issue type or label for postmortem action items and toil-reduction work, linked back to the incident that generated them, and tracked in the same sprint planning as feature work — so reliability work has to compete visibly for priority instead of being invisible 'extra' work nobody schedules.

☐ **What does 'quality analysis' mean in your day-to-day as an SRE, separate from a formal QA role?**

Reviewing release readiness against a quality bar before promotion — test coverage on the changed area, whether new alerting and dashboards exist for new functionality, whether a rollback plan exists. SRE-side quality analysis is about production-readiness, not correctness testing, which is the dev/QA team's responsibility.

☐ **How do you track and report on reliability KPIs (MTTA/MTTR/incident count) to leadership on a regular cadence?**

A recurring dashboard or report — monthly is common — trending MTTA, MTTR, incident count, and error-budget burn over time, called out against any specific initiative meant to move them, so leadership sees cause and effect rather than a raw number. Frame it in business terms — customer impact, not just minutes.

☐ **How do you handle a JIRA backlog that's grown to hundreds of stale reliability tickets?**

Triage in a batch pass — close what's genuinely obsolete, because the system it referred to no longer exists — re-prioritize what's still relevant against current risk — and resist the instinct to just let it keep growing silently, since a backlog nobody trusts stops being used for real prioritization.

☐ **What's your approach to writing a JIRA ticket for reliability work so it actually gets picked up, not deprioritized indefinitely?**

Concrete and scoped, with the 'why now' tied to a real cost — an incident it would have prevented, a burn-rate trend — rather than a vague 'improve reliability' ticket that's easy to deprioritize. Specificity and a named cost are what get reliability work funded against competing feature priorities.

☐ **How do you balance production support ticket load against planned project work as a Lead?**

Protect explicit capacity for planned/proactive work in sprint planning, rather than treating it as whatever's left after support. Rotate who's primarily fielding support that sprint, so it doesn't always fall on the same person. Treat a sustained spike in support load itself as a signal worth investigating, not just bad luck.

Leadership & Behavioral

DAY-TO-DAY, AUTOMATION & BEHAVIORAL FRAMEWORKS

☐ **Describe your day-to-day as an SRE Lead. (framework)**

Framework: a mix of reviewing overnight alerts/incidents first thing, checking error-budget and SLO dashboards, time explicitly blocked for planned reliability/automation work rather than only reactive work, architecture/design-review participation, and mentoring time with the team. The balance shifts week to week, but the discipline is protecting time for the proactive half against the reactive half.

☐ **Tell me about a time you automated away a recurring manual task. (automation narrative — framework)**

Structure: what was manual and how often it recurred → the cost — time, error rate, on-call burden → what you built → the measurable result. *Bring your own* — the verified 60% manual-deployment-reduction metric is a strong anchor here if it fits the story being asked for.

☐ **How do you decide what to automate first when everything feels like it needs automating?**

Prioritize by frequency × pain, not by technical interest — a task done daily with moderate pain often beats a rare task with high pain, because the daily compounding cost adds up faster. Also prioritize anything that's a common source of human error during incidents — a manual step performed under pressure — over convenience automation.

☐ **Tell me about a time you disagreed with a decision from above and how you handled it. (framework)**

Framework: state your reasoning once, clearly and with evidence; accept the final call once it's made, even if it's not what you'd have chosen; and if the outcome later proves the concern valid, raise it constructively in the postmortem or retro rather than as 'I told you so.' *Bring your own* real example.

☐ **How do you stay current with new technology while also owning day-to-day production stability?**

Framework: dedicate small, protected time regularly rather than waiting for a slow week that never comes; prioritize learning that's directly applicable to a current gap, not just whatever's trending; and use real production problems as the forcing function to actually go deep, rather than passive reading.

☐ **Tell me about a customer-facing moment where you had to explain a technical outage to a non-technical stakeholder. (framework)**

Framework: lead with impact and current status in plain language before any technical detail, avoid jargon that makes the situation sound scarier than it is, give a concrete next-update time, and save full root-cause detail for the postmortem rather than speculating live. *Bring your own* real example if you have one.

☐ **What's a mistake you made in production and what did you learn from it? (framework)**

Framework: pick something real and own it plainly — this is the blameless-postmortem mindset applied to yourself — state the specific system or process change that came out of it, and avoid picking something so trivial it doesn't demonstrate real learning. *Bring your own* — don't fabricate; if nothing verified comes to mind, work this one out ahead of the interview rather than inventing an example on the spot.

☐ **Where do you see yourself growing in the first 90 days of this Lead role? (framework)**

Framework: first 30 days — learn the current SLOs, architecture, and team dynamics before changing anything. Next 30 — identify the highest-leverage gap, often visibility/alerting or a recurring incident pattern, and propose a fix. Final 30 — start formalizing the team-management pieces this JD calls out: mentoring cadence, hiring involvement. Avoid promising specific outcomes before actually seeing the environment.